DOCKER EKS WORK FLOW

Dockerfile
 ↓
docker build
 ↓
Docker Image
 ↓
docker run
 ↓
Container
 ↓
docker push
 ↓
AWS ECR
 ↓
Kubernetes
 ↓
Pod
 ↓
Container

1. Dockerfile first create cheyali

Nee application project folder lo Dockerfile ane file create cheyali.
Example Java application anukundam:

```
myapp/
├── src/
├── pom.xml
└── Dockerfile
```

Dockerfile lo Docker ki instructions istam:

```
FROM eclipse-temurin:17-jre

WORKDIR /app

COPY target/myapp.jar app.jar

EXPOSE 8080

ENTRYPOINT ["java", "-jar", "app.jar"]
```

Ikkada:
FROM → application run avvadaniki base image.
WORKDIR → container lopala working directory.
COPY → mana JAR file ni container image lopalaki copy chestundi.
EXPOSE → application 8080 port lo listen chestundi ani documentation.
ENTRYPOINT → container start ayinappudu Java application run cheyyali.
Important: Dockerfile itself image kaadu. Dockerfile ante **image ela create cheyyalo cheppe instructions**.

---

2. Docker Image ela create cheyali?

First application build cheyyali.
Example:

```
mvn clean package
```

Appudu:

```
target/myapp.jar
```

vastundi.
Ippudu same project folder lo:

```
docker build -t myapp:1.0 .
```

run cheyali.
Ikkada . ante **current folder lo Dockerfile undi, akkadanunchi build cheyyi**.
Docker Dockerfile ni read chesi instructions execute chestundi.
Finally:

```
myapp:1.0
```

ane **Docker image** create avtundi.
Check:

```
docker images
```

Example:

```
REPOSITORY   TAG   IMAGE ID
myapp        1.0   abc123
```

---

3. Image ante enti?

Image ni **ready-made package/template** laga anuko.
Image lo:

- application

- required libraries
- runtime
- configuration/files

untayi.
Kani **image running application kaadu**.
Image ready ga unna template.

---

4. Container ante enti?
Image nunchi actual running instance create chesthe adi **container**.
Command:
docker run -d --name myapp-container -p 8080:8080 myapp:1.0
Ikkada Docker:
myapp:1.0 Image
↓
Container
↓
Java Application Running
Container create chesi application start chestundi.
Check:
docker ps
Example:
CONTAINER ID NAME
abc123 myapp-container
Browser:
http://localhost:8080
open chesthe application vastundi.

---

5. Image okkate, containers enni?
Oka image nunchi multiple containers create cheyyachu.
Example:
myapp:1.0
↓
├— Container 1
├— Container 2
└— Container 3
Commands:
docker run -d --name app1 myapp:1.0
docker run -d --name app2 myapp:1.0
docker run -d --name app3 myapp:1.0
Ippudu same image nunchi 3 containers run avutayi.
But real production lo **manual ga 3 containers create cheyyadam usually cheyyam**.
Kubernetes ni use chesthe Kubernetes itself required Pods/containers ni manage chestundi.

---

6. Docker image ni ECR ki enduku pampali?
Ippudu image nee laptop lo undi:
Laptop
↓
myapp:1.0
Kani production EKS cluster nee laptop lo ledu.
EKS worker nodes ki aa image kavali.
Anduke image ni **AWS ECR** lo store chestam.
First AWS ECR lo repository create chestam:
ECR Repository:
myapp
Then Docker image ki ECR tag ivvali.
Example:
docker tag myapp:1.0 123456789.dkr.ecr.ap-south-1.amazonaws.com/myapp:1.0
Ippudu same image ki ECR address tho tag vachindi.

---

7. ECR ki image ela push cheyali?
First Docker/AWS authentication cheyyali.
aws ecr get-login-password --region ap-south-1 | docker login --username AWS --password-stdin 123456789.dkr.ecr.ap-south-1.amazonaws.com
Login successful ayyaka:
docker push 123456789.dkr.ecr.ap-south-1.amazonaws.com/myapp:1.0
Ippudu image:
Laptop
↓
Docker Image
↓

AWS ECR
ECR lo store ayyindi.

---

8. Ippudu important question — ECR nunchi Kubernetes ki image ela copy avtundi?
Manam manually copy cheyyam.
Idi very important.
Kubernetes Deployment YAML lo ECR image address specify chestam.
Example:

```
containers:
  - name: myapp
    image: 123456789.dkr.ecr.ap-south-1.amazonaws.com/myapp:1.0
```

Ippudu:

```
kubectl apply -f deployment.yaml
```

ani run chestam.
Kubernetes Deployment create avtundi.
Kubernetes Scheduler oka Pod ni worker Node meeda place chestundi.
Aa Node lo container runtime ECR ki vellindi:
“Naaku myapp:1.0 image kavali.”
AWS permissions correct ga unte ECR nunchi image **automatically pull** chestundi.
So:

```
ECR
↓
Image Pull
↓
Worker Node
↓
Pod
↓
Container
↓
Application Running
```

ECR nunchi Kubernetes ki image ni nuvvu scp **or** cp **command tho copy cheyyavu. Kubernetes/container runtime image ni pull chestundi.**

---

9. Kubernetes lo container evaru create chestaru?
Nuvvu generally:

```
docker run
```

Kubernetes production lo manually run cheyyavu.
Nuvvu Deployment YAML lo desired state define chestav:

```
replicas: 3
```

Kubernetes cheptundi:
“Naaku 3 application replicas kavali.”
Deployment → ReplicaSet → Pods create/manage chestayi.
Pod create ayyaka worker node lo **kubelet** container runtime ni use chesi required container start cheyyistundi.
So production lo:

```
Deployment
  ↓
ReplicaSet
  ↓
Pod
  ↓
Container
```

---

10. Mari 3 replicas ante 3 images aa?
Kaadu.
Oka image:

```
myapp:1.0
```

ECR lo **one image**.
Kubernetes 3 replicas create chesthe:

```
ECR
 │
 │ same image
 │
 ├── Pod 1 → Container
 ├── Pod 2 → Container
 └── Pod 3 → Container
```

Three running containers same image version nunchi start avvachu.
Image duplicate ga ECR lo 3 times create cheyyalsina avasaram ledu.

---

11. Full Docker → ECR → Kubernetes flow
Ippudu complete ga:

Developer Code
↓
Maven Build
↓
target/myapp.jar
↓
Dockerfile
↓
docker build
↓
Docker Image
myapp:1.0
↓
docker tag
↓
ECR Image
↓
docker push
↓
AWS ECR
↓
Kubernetes Deployment YAML
↓
kubectl apply
↓
Deployment
↓
ReplicaSet
↓
Pods
↓
Worker Node
↓
Kubernetes asks for ECR image
↓
Image Pull
↓
Container Created
↓
Application Running
Most important difference
Docker local practice lo:
Image
↓
docker run
↓
Container
Kubernetes production lo:
ECR Image
↓
Deployment
↓
ReplicaSet
↓
Pod
↓
Container
Kubernetes **container ni manually nuvvu create cheyyavu**. Nuvvu desired configuration YAML lo istav; Kubernetes aa configuration prakaram Pods/containers ni create and maintain chestundi.
image: nginx
Ikkada **2 things define chesam**:
replicas: 3
↓
3 Pods kavali

containers:
↓
Prathi Pod lo 1 nginx container kavali
Nuvvu:
kubectl apply -f deployment.yaml
cheste request API Server ki vellindi.

---

6. Deployment → ReplicaSet
Deployment controller chusi:
"User ki 3 replicas kavali."
ani understand chestundi.
Deployment automatically **ReplicaSet** create chestundi.
Deployment
↓
ReplicaSet
ReplicaSet responsibility:
3 Pods always running undela maintain cheyyali.

---

7. ReplicaSet → Pods
ReplicaSet 3 Pods create cheyyamani Kubernetes API ki request chestundi.
ReplicaSet
↓
Pod 1
Pod 2
Pod 3
Important:
Ee point lo **Pods create ayyayi**, kani avi **ye Node meeda run avvali anedi inka decide avvaledu**.
And each Pod lo manam Deployment lo define chesina container untundi:
Pod 1 → nginx container
Pod 2 → nginx container
Pod 3 → nginx container
So total:
3 Pods
↓
3 Containers

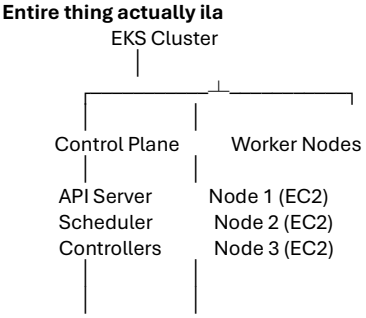
---

8. Scheduler → Node select chestundi
Ippudu Kubernetes **Scheduler** work start chestundi.
Scheduler available Nodes ni check chestundi:
Node 1 → CPU available
Node 2 → CPU available
Node 3 → CPU available
Then suitable Nodes select chestundi.
Example:
Node 1 → Pod 1
Node 2 → Pod 2
Node 3 → Pod 3

---

9. Pod lo Containers run avuthayi
Pod Node meeda assign ayyaka, aa Node lo unna **kubelet** container runtime ni use chesi container ni start chestundi.
Pod
↓
Container
↓
nginx application running
So final ga:
Node 1
↓
Pod 1
↓
nginx Container
↓
Application
Same way Node 2 and Node 3 lo kuda.

Entire thing actually ila



Deployment Pods
↓ ↓
ReplicaSet Containers
↓ ↓
Pods Application
│
↓
Scheduler
↓
Suitable Node

One-line lo complete flow:
Cluster → Node Group → EC2 → Kubernetes Nodes → Deployment → ReplicaSet → Pods → Scheduler → Nodes → Containers → Application

Gurthupettuko:
Nodes = Machines
Deployment = Application desired state
ReplicaSet = Required number of Pods maintain chestundi
Pods = Application run ayye unit
Containers = Pod lopala actual application run avtundi

Pods + Containers run ayyaka actual application traffic process start avtundi. Akkada nunchi next flow ila untundi:

Cluster
↓
Nodes
↓
Deployment
↓
ReplicaSet
↓
Pods
↓
Containers
↓
Service
↓
Ingress / Load Balancer
↓
User / Client

10. Service create chestam
Pods direct ga user ki expose cheyyadam usually cheyyamu.
Example:
kind: Service
Service ki cheptham:
"Ee application Pods ni expose cheyyi."
Service labels/selectors use chesi correct Pods ni identify chestundi.

Service
↓
Pod 1
Pod 2
Pod 3

---

11. User request vastundi
Suppose user browser lo:
https://myapp.com
open chesadu.
Request first **Load Balancer / Ingress** daggaraki vastundi.

User
↓
Load Balancer
↓
Ingress
↓
Service

---

12. Service → Pod
Service available Pods lo oka Pod ki request forward chestundi.

Service
↓
Pod 1
Pod 1 lo:
Container
↓

Application
request process chestundi.

---

13. Response back to User
Application response istundi:
Application
↓
Container
↓
Pod
↓
Service
↓
Ingress / Load Balancer
↓
User
So complete production flow:
1. Cluster
↓
2. Node Group
↓
3. EC2
↓
4. Kubernetes Nodes
↓
5. Deployment
↓
6. ReplicaSet
↓
7. Pods
↓
8. Scheduler
↓
9. Containers
↓
10. Service
↓
11. Ingress / Load Balancer
↓
12. User Request
↓
13. Pod processes request
↓
14. Response → User
But production lo story ikkada end kaadu. Next important concepts **ConfigMap, Secret, Storage, Ingress, HPA, monitoring, logging, CI/CD** etc. vastayi.
_____-

User request → Pod → response ayyaka, production lo Kubernetes continuous ga application ni **monitor & maintain** chestu untundi.
Next process:
1. User Request
↓
2. Load Balancer / Ingress
↓
3. Service
↓
4. Pod
↓
5. Container
↓
6. Application Response
↓
7. Monitoring
↓
8. Health Checks
↓
9. Failure ayithe → Pod restart / new Pod
↓
10. Traffic perigite → HPA → More Pods
↓

11. Pods ki capacity saripokapothe → More Nodes

15. Health check

Kubernetes continuously check chestundi:

"Application healthy ga unda?"

For example:

Pod 1 → Healthy ✅

Pod 2 → Healthy ✅

Pod 3 → Failed ❌

Failed Pod unte Kubernetes desired replicas maintain cheyyadaniki replacement Pod create chestundi.

Pod 3 ❌

↓

New Pod 4 ✅

16. Traffic perigite

Suppose normal ga:

3 Pods

Traffic suddenly perigindi:

100 users → 10,000 users

HPA (Horizontal Pod Autoscaler) configure chesi unte:

3 Pods

↓

5 Pods

↓

10 Pods

ani scale avvachu.

17. Nodes capacity saripokapothe

Suppose existing nodes full:

Node 1 → 90% CPU

Node 2 → 95% CPU

Node 3 → 92% CPU

New Pods ki space lekapothe, **Cluster Autoscaler / Karpenter** lanti mechanism new EC2 nodes add cheyyagaladu.

3 Nodes

↓

5 Nodes

↓

New Pods schedule

Final production cycle

User

↓

Load Balancer / Ingress

↓

Service

↓

Pods

↓

Containers

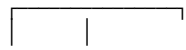
↓

Application

↓

Monitoring + Health Checks

↓



Failure More Traffic

↓ ↓

New Pod More Pods

↓

No Node Capacity

↓

More Nodes

So Kubernetes oka sari Pods create chesi vadileyadu. Desired state maintain cheyyadaniki **continuously observe → compare → fix/scale** chestu untundi.

---

inka next production side lo actual DevOps cycle start avtundi.

Application running lo undi anukundam. Next:

1. Developer code change chestadu

↓

2. Git lo code push chestadu

↓

3. CI/CD Pipeline trigger avtundi

↓

4. Application build avtundi
↓
5. Docker Image create avtundi
↓
6. Image → Container Registry (ECR)
↓
7. Kubernetes Deployment update avtundi
↓
8. New Pods create avtayi
↓
9. Rolling Update jarugutundi
↓
10. Old Pods gradually terminate avtayi
↓
11. New Pods traffic receive chestayi
↓
12. Monitoring / Logs / Alerts

Example

Developer code change chesadu:
app v1
↓
code change
↓
app v2
CI/CD:
Git
↓
Jenkins / GitHub Actions
↓
Docker Build
↓
ECR
↓
Kubernetes
↓
Deployment
Kubernetes lo:
Old:
Pod 1 → v1
Pod 2 → v1
Pod 3 → v1

↓ Rolling Update

New:
Pod 1 → v2
Pod 2 → v2
Pod 3 → v2

Important: Production lo usually application update kosam **cluster ni recreate cheyyaru**. Existing cluster lo **Deployment update → new Pods → rolling update** jarugutundi.

So complete real-time picture:
Infrastructure
↓
Cluster
↓
Nodes
↓
Deployment
↓
ReplicaSet
↓
Pods
↓
Containers
↓
Service
↓
Ingress / Load Balancer
↓
Users
↓

Monitoring
↓
Developer changes code
↓
Git
↓
CI/CD
↓
Docker Image
↓
ECR
↓
Deployment update
↓
New Pods
↓
Rolling Update
↓
Users get new version
Idi production lo continuous cycle. 🔁

PART 1 — Application nunchi EKS lo Running varaku
1. Developer Application Code Raastadu
First developer application create chestadu.
Example ki Java application anukundam.
Developer daggara:

```
myapp/
├── src/
├── pom.xml
└── application.properties
```

untayi.
src lo actual Java code untundi.
pom.xml lo application ki kavalsina dependencies and build information untayi.
Developer first local machine lo application run chesi test chestadu.
Example:
Developer
↓
Java Code
↓
Local Machine
↓
Application Run
Application correct ga work chestundi ani developer confirm chestadu.
Appudu aa code ni Git repository ki push chestadu.

---

2. Developer Code ni Git ki Push Chestadu
Developer:

```
git add .
git commit -m "new feature"
git push
```

chestadu.
Ippudu developer local machine lo unna code Git repository ki vellipothundi.
Git repository lo code version maintain avtundi.
Example:
Git Repository

```
main
 │
 ├── commit 1 → v1
 ├── commit 2 → v1.1
 └── commit 3 → v2
```

Ante developer future lo em change chesado kuda track cheyyachu.
Example:
Developer today v1 push chesadu.
Next day login feature add chesi v2 push chesadu.
Git daggara rendu versions history untundi.
Production lo exact ga e code deploy chesamo kuda identify cheyyachu.

---

3. Git Push Ayina Tarvatha CI/CD Pipeline Trigger Avtundi
Developer git push chesadu.
Git repository lo CI/CD integration configure chesi unte, new code vachindani pipeline ki information velthundi.
Example:
Developer
↓
git push
↓
Git
↓
CI/CD Pipeline
Pipeline ante application ni manually build/test/deploy cheyyakunda automatic ga process chese system.
Example tools:
Jenkins
GitHub Actions
GitLab CI
AWS CodePipeline
Pipeline ki basically responsibility:
"New code vachindi. Code teesuko → build cheyyi → test cheyyi → Docker image create cheyyi → image registry ki push cheyyi → Kubernetes lo deploy cheyyi."
So pipeline stages ila untayi:
Checkout Code
↓
Build
↓
Test
↓
Docker Build
↓
Docker Image
↓
ECR
↓
Deploy to EKS

---

4. CI Pipeline Code ni Build Chestundi
Suppose mana application Java application.
Pipeline Git nunchi code teesukoni build chestundi.
Example:
mvn clean package
run chestundi.
Maven source code ni compile chesi application package chestundi.
Build successful ayyaka:
target/
↓
app.jar
vastundi.
Tests kuda run avtayi.
Suppose application lo test fail ayindi.
Build
↓
Test ❌
↓
Pipeline STOP
Appudu Docker image create cheyyadu.
Production ki deploy kuda cheyyadu.
Tests successful ayithe:
Build ✅
↓
Test ✅
↓
Docker Build
next stage ki velthundi.

---

5. Docker Image Create Chestundi
Ippudu mana daggara:
app.jar
undi.
Kani Kubernetes lo application ni container ga run cheyyali.
Anduke Docker image create chestam.
Example Dockerfile:

```
FROM openjdk:17
COPY target/app.jar app.jar
CMD ["java", "-jar", "app.jar"]
```

Ikkada Dockerfile chepthundi:

- Java runtime kavali.
- Mana app.jar image lopalaki copy cheyyali.
- Container start ayinappudu Java application run cheyyali.

Pipeline:
docker build -t myapp:v1 .
run chestundi.
Ippudu:
myapp:v1
ane Docker image create avtundi.
Image ante simple ga cheppalante application ni container lo run cheyyadaniki required package.
Image lo:
Application
+
Runtime
+
Required Dependencies
+
Required Files
package ayi untayi.

---

6. Docker Image ni ECR ki Push Chestam
Ippudu Docker image local pipeline machine daggara undi.
Kani Kubernetes worker node ki later ee image kavali.
Kabatti image ni central container registry lo store chestam.
AWS environment lo commonly **ECR (Elastic Container Registry)** use chestam.
Flow:
Docker Image
↓
ECR
Example:
ECR Repository

```
myapp
 ├── v1
 ├── v2
 └── v3
```

Pipeline:
docker push <ECR>/myapp:v1
chestundi.
Ippudu myapp:v1 image ECR lo safe ga store ayi undi.
Later Kubernetes Pod create ayinappudu:
"Naaku <ECR>/myapp:v1 image kavali."
ani Node ki cheppagaladu.

PART 2 — EKS lo Application Run Avvadam
7. EKS Cluster Already Untundi
Production lo developer prathi application deployment kosam new EKS cluster create cheyyadu.
Usually infrastructure already create chesi untundi.
Example:
EKS Cluster
↓
Control Plane
+
Worker Nodes
Cluster ante Kubernetes environment.
Cluster lo main ga rendu sides imagine cheyyachu:

```
EKS Cluster
   │
   ├── Control Plane
   │
   └── Worker Nodes
```

Control Plane cluster ni control/manage chestundi.
Worker Nodes actual application workloads ni run chestayi.

---

8. Control Plane Em Chestundi?
Control Plane Kubernetes ki brain laga work chestundi.

Dantlo important components untayi.
API Server
API Server Kubernetes ki main entry point.
Manam:
kubectl apply -f deployment.yaml
cheste mana request first API Server ki velthundi.
API Server request ni accept chesi Kubernetes cluster state lo record chestundi.
Scheduler
Scheduler responsibility:
"Create ayina Pod ye Node meeda run avvali?"
ani decide cheyyadam.
Scheduler container ni direct ga start cheyyadu.
Node selection matrame chestundi.
Controller Manager
Controllers desired state and actual state compare chestayi.
Example:
Desired = 3 Pods
Actual = 2 Pods
Controller:
"Oka Pod takkuva undi, required state ki teesukellali."
ani action initiate chestundi.
etcd
Cluster ki related state information store chestundi.
Example:
Nodes
Pods
Deployments
Services
Configuration
lanti Kubernetes state information etcd lo maintain avtundi.

---

9. Worker Nodes Already Available
Suppose mana production EKS cluster lo 3 worker nodes unnayi.
Node 1 → EC2
Node 2 → EC2
Node 3 → EC2
Ee Nodes actually EC2 machines.
Node lo kubelet untundi.
Kubelet Node taraf nunchi Kubernetes instructions follow chestundi.
Example:
"Pod 1 ni ee Node meeda run cheyyali."
ani Scheduler decision vachaka, aa Node lo kubelet aa Pod ni run cheyyadaniki required work chestundi.

---

10. Deployment Create Chestam
Ippudu ECR lo:
myapp:v1
image ready ga undi.
EKS lo application run cheyyali.
Anduke Deployment YAML create chestam.
Example:

```
apiVersion: apps/v1
kind: Deployment

metadata:
  name: myapp

spec:
  replicas: 3

  selector:
    matchLabels:
      app: myapp

  template:
    metadata:
      labels:
        app: myapp

    spec:
      containers:
      - name: myapp
```

image: <ECR>/myapp:v1
Ikkada rendu very important things define chesam.
replicas: 3
Meaning:
"Na application ki 3 copies kavali."
So Kubernetes ultimately:
Pod 1
Pod 2
Pod 3
maintain cheyyali.
containers
Ikkada:
image: <ECR>/myapp:v1
icham.
Meaning:
"Prathi Pod lo myapp:v1 image tho container run cheyyali."
So:
3 Pods
×
1 Container per Pod
=
3 Containers

---

11. kubectl apply **Chestam**
Ippudu Deployment YAML ready.
Manam:
kubectl apply -f deployment.yaml
run chestam.
kubectl request ni Kubernetes API Server ki pampistundi.
kubectl
↓
API Server
API Server Deployment definition ni accept chestundi.
Ippudu Kubernetes ki user desired state telisindi:
Application = myapp
Replicas = 3
Image = myapp:v1
Ante:
"Naaku myapp application 3 replicas tho run avvali."
ani Kubernetes ki cheppam.

---

12. Deployment → ReplicaSet
Ippudu Deployment controller work start chestundi.
Deployment observe chesi:
Desired:
3 Pods
ani identify chestundi.
Deployment direct ga Pods ni manage cheyyadam kanna ReplicaSet ni create/manage chestundi.
So:
Deployment
↓
ReplicaSet
ReplicaSet ki responsibility:
"Naaku Deployment cheppina number of Pods always maintain cheyyali."
Example:
Desired = 3
ReplicaSet responsibility:
3 Pods
available ga undela maintain cheyyadam.

---

13. ReplicaSet → Pods Create Chestundi
Ippudu ReplicaSet check chestundi:
Desired Pods = 3
Actual Pods = 0
Difference:
3 - 0 = 3
Kabatti ReplicaSet Kubernetes API ki 3 Pods create cheyyadaniki request chestundi.
Ippudu:
ReplicaSet
↓
Pod 1

Pod 2
Pod 3
create avtayi.
Important point:
Pods Kubernetes objects ga create ayyayi.
Kani ippudu inka Scheduler:
"Ee Pod ye Node meeda run avvali?"
ani decide cheyyali.

---

14. Prathi Pod lo Container Definition Untundi
Deployment YAML lo already:
containers:
- name: myapp
  image: <ECR>/myapp:v1
define chesam.
Kabatti create ayina Pod template prakaram prathi Pod lo container definition untundi.
Pod 1
↓
myapp container

Pod 2
↓
myapp container

Pod 3
↓
myapp container
So mana configuration:
replicas = 3
containers per Pod = 1
Kabatti:
3 Pods
↓
3 Containers
Kani **container image undi ani container actual application already run avtundi ani kaadu**.
Actual container start cheyyadaniki Pod ki Node assign avvali.

---

15. Scheduler → Node Select Chestundi
Ippudu Scheduler observe chestundi.
Pods create ayyayi.
Kani avi Node assignment lekunda unnayi.
Scheduler available Nodes ni check chestundi.
Suppose:
Node 1 → CPU available
Node 2 → CPU available
Node 3 → CPU available
Scheduler Pod requirements ni Nodes tho compare chestundi.
CPU/memory available unda?
Node suitable aa?
Taints/tolerations, affinity, constraints etc. unnaya?
ani consider chestundi.
Suppose suitable Nodes:
Pod 1 → Node 1
Pod 2 → Node 2
Pod 3 → Node 3
Scheduler ee assignment ni Kubernetes API Server ki record chestundi.
Important:
Scheduler actual container start cheyyadu.
Scheduler only:
"Pod 1 Node 1 meeda run avvali."
ani decision chestundi.

---

16. Kubelet → Container Runtime → ECR Image → Container
Ippudu Pod 1 ki:
Node 1
assign ayyindi.
Node 1 lo kubelet ki aa information telustundi.
Kubelet:
"Pod 1 na Node meeda run avvali."
ani identify chestundi.
Ippudu kubelet container runtime ni use chestundi.

Modern Kubernetes environments lo commonly containerd lanti runtime untundi.
Container runtime responsibility:
"Container ni actual ga create chesi run cheyyali."
Kani container create cheyyadaniki image kavali.
Deployment lo image:
<ECR>/myapp:v1
ani icham.
So runtime/node daggara image already local ga unte reuse cheyyachu.
Image local ga lekapothe registry nunchi pull chestundi.
ECR
↓
myapp:v1
↓
Node 1
↓
Container Runtime
Image Node ki vachaka container runtime aa image tho container create/start chestundi.
Node 1
↓
Pod 1
↓
Container
↓
myapp:v1
↓
Application Running
Same process Node 2 and Node 3 lo kuda jarugutundi.
Final ga:
Node 1
↓
Pod 1
↓
Container
↓
myapp application
Node 2
↓
Pod 2
↓
Container
↓
myapp application
Node 3
↓
Pod 3
↓
Container
↓
myapp application
Ippudu actual application **3 Pods lo run avtundi**.

Ikkadi Varaku Complete Flow
Developer
↓
Application Code
↓
Git
↓
CI/CD
↓
Build + Test
↓
Docker Image
↓
ECR
↓
Existing EKS Cluster
↓
Deployment
↓
ReplicaSet

↓
Pods
↓
Scheduler
↓
Nodes
↓
Kubelet
↓
Container Runtime
↓
ECR Image
↓
Container
↓
Application
Ippudu application **run avtundi**.
Kani user inka direct ga Pod ni access cheyyadam start cheyyaledu.
Akkada nunchi next part: Service.

PART 2 — Application ni User ki Expose Cheyyadam
17. Pods Direct ga User ki Enduku Ivvakudadhu?
Suppose mana Pods:
Pod 1 → 10.0.1.10
Pod 2 → 10.0.1.11
Pod 3 → 10.0.1.12
unnayi.
User direct ga:
10.0.1.10
ki request pampiste work avvachu.
Kani Pod permanent machine kaadu.
Suppose Pod 2 fail ayindi.
Pod 2 ❌
ReplicaSet replacement Pod create chestundi:
Pod 4 → 10.0.1.25
Ippudu old Pod IP:
10.0.1.11
available kaadu.
Kabatti user ki direct Pod IP ivvadam reliable kaadu.
Manaki stable endpoint kavali.
Anduke **Service** create chestam.

---

18. Service Create Chestam
Service ki manam cheptham:
"Na application Pods ni identify chesi, vatiki stable network endpoint ivvu."
Example:

```
kind: Service

selector:
  app: myapp
```

Mana Pods lo:
label:
app=myapp
untundi.
Service selector use chesi matching Pods ni identify chestundi.
So:

```
Service
   │
   ├── Pod 1
   ├── Pod 2
   └── Pod 3
```

Pod 2 fail ayithe:

```
Service
   │
   ├── Pod 1
   ├── Pod 3
   └── Pod 4
```

Service automatically current healthy/eligible Pods ni backend ga use cheyyagaladu.
Kabatti user ki individual Pod IP gurinchi teliyalsina avasaram ledu.

---

19. User Request Vastundi

Suppose user browser lo:
https://myapp.com
open chestadu.
Request internet nunchi AWS environment ki vastundi.
Typical production flow:
User
↓
DNS
↓
Load Balancer
↓
Ingress
↓
Service
↓
Pod
↓
Container
↓
Application
Ikkada DNS myapp.com ni correct external endpoint ki resolve cheyyadaniki use avtundi.

---

20. Load Balancer / Ingress Request ni Route Chestayi
External request cluster loki ravali.
AWS Load Balancer external traffic receive cheyyagaladu.
Ingress rules request ni correct application/service ki route cheyyagalavu.
Example:
myapp.com
↓
myapp Service
Another application:
payment.myapp.com
↓
payment Service
So Ingress main ga:
"Ee request ye Service ki vellali?"
ani decide chestundi.

---

21. Service → Correct Pod
Request Service ki vachindi.
Service daggara matching Pods list untundi.
Example:
Service
│
├── Pod 1
├── Pod 2
└── Pod 3
Service appropriate backend Pod ki request forward chestundi.
Example:
Request 1 → Pod 1
Request 2 → Pod 2
Request 3 → Pod 3
Exact distribution networking/configuration batti vary avvachu.

---

22. Pod → Container → Application
Suppose request Pod 1 ki vellindi.
Pod 1 lo actual container:
Pod 1
↓
myapp Container
↓
myapp Application
Application request process chestundi.
Example:
GET /users
Application:
Request
↓
Application Code
↓
Business Logic

↓
Database/API
↓
Response
generate chestundi.

---

23. Response User ki Back Veltundi
Application response create chesindi.
Response reverse direction lo velthundi:
Application
↓
Container
↓
Pod
↓
Service
↓
Ingress
↓
Load Balancer
↓
User
So actual request-response:
User
↓
Load Balancer
↓
Ingress
↓
Service
↓
Pod
↓
Container
↓
Application
↓
Response
↓
User
Ippudu application user requests serve chestundi.

---

24. Kubernetes Continuous ga Application ni Maintain Chestundi
Application run ayyaka Kubernetes:
"Naa work complete."
ani stop avvadu.
Kubernetes continuously desired state maintain chestundi.
Mana Deployment lo:
replicas: 3
undi.
Meaning:
"Naaku 3 application Pods kavali."
Normal situation:
Pod 1 → Healthy
Pod 2 → Healthy
Pod 3 → Healthy
Kani Pod 2 fail ayithe:
Pod 1 → Healthy
Pod 2 → Failed
Pod 3 → Healthy
Ippudu actual:
2 Pods
Desired:
3 Pods
ReplicaSet difference ni identify chesi replacement Pod create chestundi.
Pod 2 ❌
↓
Replacement Pod
↓
Pod 4
Scheduler Pod 4 ki Node select chestundi.

Kubelet container start chestundi.
Finally:
Pod 1 → Healthy
Pod 3 → Healthy
Pod 4 → Healthy
Malli 3 Pods.

---

25. Health Checks
Application running lo unna, actually healthy ga unda ani check cheyyadaniki probes use chestam.
Readiness Probe
Readiness meaning:
"Ee Pod ippudu traffic receive cheyyadaniki ready aa?"
Suppose:
Pod 1 → Ready
Pod 2 → Starting
Pod 3 → Ready
Service Pod 2 ki traffic ivvakunda wait cheyyachu.
Application fully ready ayyaka:
Pod 2 → Ready
appudu traffic receive cheyyagaladu.
Liveness Probe
Liveness meaning:
"Application alive ga unda?"
Suppose container running state lo undi, kani application hang ayindi.
Liveness probe fail ayithe configured behavior prakaram kubelet container restart cheyyagaladu.
So:
Readiness
→ Traffic receive cheyyadaniki ready aa?

Liveness
→ Application alive ga unda?

---

26. Traffic Perigite HPA
Suppose normal ga:
3 Pods
running unnayi.
Suddenly traffic:
100 users
↓
1,000 users
↓
10,000 users
ayindi.
Existing Pods meeda load perigindi.
Example:
Pod 1 → CPU 85%
Pod 2 → CPU 90%
Pod 3 → CPU 88%
HPA configure chesi unte metrics base chesukoni more replicas kavali ani decide cheyyachu.
HPA
↓
3 Pods
↓
5 Pods
↓
10 Pods
HPA itself EC2 machines create cheyyadu.
It first **Pod replicas** ni scale chestundi.

---

27. More Pods → Node Capacity Problem
Suppose HPA 10 Pods kavali ani cheppindi.
Kani existing Nodes already full:
Node 1 → 90%
Node 2 → 95%
Node 3 → 92%
New Pods create ayina, suitable Node capacity lekapothe avi:
Pod 8 → Pending
Pod 9 → Pending
Pod 10 → Pending
state lo undachu.
Ippudu node autoscaling mechanism configured unte:

Cluster Autoscaler
or
Karpenter
new EC2 capacity add cheyyagaladu.

---

28. New EC2 → New Kubernetes Nodes
Suppose already:
Node 1
Node 2
Node 3
unnayi.
Capacity kosam:
EC2 4
EC2 5
create avvachu.
Avi EKS cluster lo Nodes ga join avuthayi.
EC2 4
↓
Node 4

EC2 5
↓
Node 5
Nodes ready ayyaka Scheduler pending Pods ni aa Nodes meeda place chestundi.
Pending Pod
↓
Scheduler
↓
New Node
↓
Kubelet
↓
Container
↓
Application
So important distinction:
More Traffic
↓
HPA
↓
More Pods

More Pods
↓
Node Capacity saripoledu
↓
Node Autoscaling
↓
More EC2 Instances
↓
More Kubernetes Nodes

---

29. Developer New Version Create Chestadu
Application production lo v1 run avtundi.
Developer new feature add chestadu.
v1
↓
Code Change
↓
v2
Developer code Git ki push chestadu.
Developer
↓
Git Push
↓
CI/CD
Pipeline malli:
Build
↓
Test
↓

Docker Build
↓
myapp:v2
↓
ECR
create chestundi.
Ippudu ECR lo:
myapp
├── v1
└── v2
untayi.

---

30. Deployment Image Update Chestam
Production lo **new version kosam cluster recreate cheyyamu**.
Existing:
EKS Cluster
↓
Nodes
↓
Deployment
same ga continue avtayi.
Deployment lo image:
myapp:v1
nunchi:
myapp:v2
ki update chestam.
CI/CD ee update automatic ga cheyyachu.

---

31. New ReplicaSet Create Avtundi
Deployment new image version ni represent cheyyadaniki new ReplicaSet create/manage chestundi.
Old:
ReplicaSet-v1
↓
v1 Pods
New:
ReplicaSet-v2
↓
v2 Pods
Deployment rendu ReplicaSets ni manage chestundi while rollout happens.
Deployment
│
├── ReplicaSet-v1
│ ↓
│ v1 Pods
│
└── ReplicaSet-v2
↓
v2 Pods

---

32. Rolling Update Jarugutundi
Current production:
Pod 1 → v1
Pod 2 → v1
Pod 3 → v1
Kubernetes new version ni gradual ga introduce chestundi.
First:
Pod 1 → v2
Pod 2 → v1
Pod 3 → v1
Tarvatha:
Pod 1 → v2
Pod 2 → v2
Pod 3 → v1
Finally:
Pod 1 → v2
Pod 2 → v2
Pod 3 → v2
Old v1 Pods gradually terminate avtayi.
New v2 Pods Ready ayyaka traffic receive chestayi.

---

33. Monitoring

Production lo application running undi ani matrame saripodu.
Manam continuously:
CPU
Memory
Pod Health
Node Health
Request Count
Response Time
Errors
Logs
monitor chestam.
Example:
Application
↓
Metrics + Logs
↓
Monitoring System
↓
Alert
↓
DevOps / SRE
Common tools:
CloudWatch
Prometheus
Grafana
OpenSearch / ELK
Problem vachinappudu monitoring system alert generate cheyyagaladu.

COMPLETE REAL-TIME PRODUCTION FLOW
Developer
↓
Application Code
↓
Git
↓
CI/CD
↓
Build
↓
Test
↓
Docker Image
↓
ECR
↓
Existing EKS Cluster
↓
Deployment
↓
ReplicaSet
↓
Pods
↓
Scheduler
↓
Kubernetes Nodes
↓
Kubelet
↓
Container Runtime
↓
ECR Image Pull
↓
Container
↓
Application
↓
Service
↓
Ingress / Load Balancer
↓

User
User request vachinappudu:
User
↓
Load Balancer
↓
Ingress
↓
Service
↓
Pod
↓
Container
↓
Application
↓
Response
↓
User
Application running lo unnappudu Kubernetes side continuous ga:
Health Check
↓
Healthy?
/ \
Yes No
↓ ↓
Continue Restart /
Replacement Pod
Traffic perigite:
Traffic ↑
↓
HPA
↓
More Pods
↓
Scheduler
↓
Available Nodes
Nodes capacity saripokapothe:
More Pods
↓
No Node Capacity
↓
Node Autoscaling
↓
New EC2
↓
New Kubernetes Node
↓
Scheduler
↓
Pending Pod
↓
Container
↓
Application
Developer new code release chesthe:
Developer
↓
Git
↓
CI/CD
↓
Build + Test
↓
Docker Image v2
↓
ECR
↓
Deployment Update
↓

New ReplicaSet
↓
New Pods
↓
Scheduler
↓
Nodes
↓
Containers
↓
Readiness Check
↓
Rolling Update
↓
Old Pods gradually removed
↓
New Pods traffic receive chestayi
↓
Monitoring

Final ga gurthupettuko

Developer
↓
Git
↓
CI/CD
↓
Docker Image
↓
ECR
↓
EKS
↓
Deployment
↓
ReplicaSet
↓
Pods
↓
Scheduler
↓
Nodes
↓
Kubelet
↓
Container Runtime
↓
Container
↓
Application
↓
Service
↓
Ingress / Load Balancer
↓
User

Kubernetes side lo main logic:

Deployment
↓
ReplicaSet
↓
Pods
↓
Scheduler
↓
Node
↓
Kubelet
↓
Container Runtime
↓
Container

↓
Application
Production lo adi complete ayyaka kuda Kubernetes stop avvadu:
Application
↓
Health Checks
↓
Monitoring
↓
Failure → Replacement Pod
↓
Traffic ↑ → HPA → More Pods
↓
Capacity ledu → More Nodes
↓
Developer Code Change
↓
CI/CD → ECR → Deployment Update
↓
Rolling Update
↓
New Version
Idi **actual end-to-end EKS + Kubernetes + DevOps production cycle**.